

GSAP ScrollTrigger — Full Website Implementation Plan

GSAP ScrollTrigger — Full Website Implementation Plan (Production-Grade Architecture)

Document Type: Technical Architecture & Implementation Blueprint **Target Project:** Takween Connect / HalalBusiness (/Users/pc/HalalBusiness) **Tech Stack:** Next.js (App Router), OpenNext for Cloudflare Workers, Cloudflare D1 Database, Vanilla CSS Tokens **Status:** Specification only — **do not implement** until explicitly instructed to proceed. Ready for phased rollout when approved.

1. Executive Summary & Architectural Foundations

This specification details the end-to-end transformation of the Takween Connect landing page into a state-of-the-art, 60–120 FPS scroll-driven web experience powered by GSAP (GreenSock Animation Platform) and ScrollTrigger.

1.1 Architecture & Runtime Realities

Next.js Server Component Boundaries: Most existing sections (Hero.tsx , ProblemSection.tsx , ValueComparison.tsx , LimitedCapacity.tsx , etc.) are React Server Components (RSC) rendered without "use client" . - Rule: Server components must remain server components where data fetching occurs (e.g., LimitedCapacity.tsx querying Cloudflare D1 via getAcceptedCountThisMonth()). - Pattern: We introduce dedicated client-side wrappers (GSAPReveal.tsx , ScrollHeader.tsx , CapacityRingAnimated.tsx , FAQAnimated.tsx , HeroMapAnimation.tsx) to execute client-only GSAP animations without forcing data-fetching logic to the client.

Edge Compatibility (OpenNext / Cloudflare Workers): The project builds via @opennextjs/cloudflare targeting V8 Worker isolates. GSAP and ScrollTrigger reference browser globals (window , document , navigator). - All GSAP initialization is isolated to client components wrapped in useGSAP or client lifecycle hooks. Zero GSAP code touches server-side bundle evaluation.

No Flash of Unstyled Content (FOUC): All elements maintain clean standard fallback CSS layouts. GSAP applies initial states via gsap.set() inside useGSAP context or CSS class states with visibility: hidden flipping to visible on ready, preventing visual jumps.

Design Token Fidelity: All tweened values (colors, shadows, font transitions) directly consume the design tokens defined in tokens.css and globals.css (e.g., --color-primary-accessible: #047857 , --color-accent: #22c55e).

Note (added when this plan was filed away): the plan references a few tokens/files (`--color-brand-midnight` , `hero-map-bg.svg` , the 17 `bg-*.svg` assets, a 4-step `ProcessSteps`) that should be re-verified against the actual codebase state at execution time, not assumed still accurate — this spec was written as a forward-looking blueprint, not a snapshot audit at build time.

2. Complete Codebase Audit & Component Mapping Matrix

Component	File Path	Current Render	Current Motion
SiteHeader	components/shared/SiteHeader.tsx	Server	CSS sticky top, static
Hero	components/sections/Hero.tsx	Server	Radial gradient, static layout
hero-map-bg.svg	public/images/hero-map-bg.svg	Static SVG asset	Static CSS background
ProblemSection	components/sections/ProblemSection.tsx	Server	Static cards, search bar mockup
ProblemSolution	components/sections/ProblemSolution.tsx	Server	Static Before/After columns
ValueComparison	components/sections/ValueComparison.tsx	Server	Static grid, clamp fonts
BuiltToFeel	components/sections/BuiltToFeel.tsx	Server	3-card grid with SVG icons
WhySponsored	components/sections/WhySponsored.tsx	Server	Dark green gradient, static panels

Component	File Path	Current Render	Current Motion
ScopeComparison	components/sections/ScopeComparison.tsx	Server	Static lists with check/plus glyphs
AfterLaunchCards	components/sections/AfterLaunchCards.tsx	Client ("use client")	React useState hover lift
QualificationFit	components/sections/QualificationFit.tsx	Server	2-column checklist
LimitedCapacity	components/sections/LimitedCapacity.tsx	Async server (D1 query)	Passes data to CapacityRing
CapacityRing	components/sections/CapacityRing.tsx	Server	CSS @keyframes ui-capacity-ring
ProcessSteps	components/sections/ProcessSteps.tsx	Client ("use client")	React hover on steps
FAQ	components/sections/FAQ.tsx	Server	Native <details> / <summary> CSS
FinalCta	components/sections/FinalCta.tsx	Server	Deep green background
Footer	components/shared/Footer.tsx	Server	Static white bar, flex layout
Reveal.tsx	components/shared/Reveal.tsx	Client	IntersectionObserver + CSS class
Logo.tsx	components/shared/Logo.tsx	Server	CSS ui-logo-spin (24s infinite)
PortfolioAndTestimonials	components/sections/PortfolioAndTestimonials.tsx	Server	Returns null (empty data)
StatsBar.tsx	components/sections/StatsBar.tsx	Server	Not currently placed in page.tsx

3. Core Technical Infrastructure (New Files)

3.1 Dependencies Required

```
npm install gsap @gsap/react @studio-freight/lenis
```

(Note: Lenis provides 60/120Hz smooth scrolling synchronized with ScrollTrigger via `ScrollTrigger.update()`.)

3.2 GSAP Registration Singleton (`lib/gsap.ts`)

Creates a unified registration point, prevents double-registration during Next.js Fast Refresh, and encapsulates accessibility checks:

```
// lib/gsap.ts
import gsap from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';

if (typeof window !== 'undefined') {
  gsap.registerPlugin(ScrollTrigger);

  gsap.defaults({
    ease: 'power3.out',
    duration: 0.8,
  });

  ScrollTrigger.defaults({
    markers: process.env.NODE_ENV === 'development' && false, // set true for debugging
  });
}

export const prefersReducedMotion = (): boolean => {
  if (typeof window === 'undefined') return false;
  return window.matchMedia('(prefers-reduced-motion: reduce)').matches;
};

export { gsap, ScrollTrigger };
```

3.3 Universal Scroll Reveal Wrapper (`components/shared/GSAPReveal.tsx`)

Replaces the simplistic `Reveal.tsx` (CSS opacity/translateY transition) with a high-performance, GPU-accelerated GSAP component:

```

// components/shared/GSAPReveal.tsx
'use client';
import React, { useRef } from 'react';
import { useGSAP } from '@gsap/react';
import { gsap, prefersReducedMotion } from '@lib/gsap';

export type RevealVariant =
  | 'fade-up'
  | 'fade-down'
  | 'slide-left'
  | 'slide-right'
  | 'scale-up'
  | 'stagger-children'
  | 'card-3d';

interface GSAPRevealProps {
  children: React.ReactNode;
  variant?: RevealVariant;
  delay?: number;
  duration?: number;
  stagger?: number;
  scrub?: boolean | number;
  className?: string;
  start?: string;
  id?: string;
}

export function GSAPReveal({
  children,
  variant = 'fade-up',
  delay = 0,
  duration = 0.85,
  stagger = 0.12,
  scrub = false,
  className = '',
  start = 'top 85%',
  id,
}: GSAPRevealProps) {
  const containerRef = useRef<HTMLDivElement>(null);

  useGSAP(() => {
    if (prefersReducedMotion() || !containerRef.current) return;
    const el = containerRef.current;

    let fromVars: gsap.TweenVars = { opacity: 0 };
    switch (variant) {
      case 'fade-up':
        fromVars = { opacity: 0, y: 36 };
        break;
      case 'fade-down':
        fromVars = { opacity: 0, y: -36 };
        break;
      case 'slide-left':
        fromVars = { opacity: 0, x: -48 };
        break;
      case 'slide-right':
        fromVars = { opacity: 0, x: 48 };
        break;
      case 'scale-up':
        fromVars = { opacity: 0, scale: 0.92 };
    }
  }, {
    delay,
    duration,
    stagger,
    scrub,
    className,
    start,
    id,
  });
}

```

```

        break;
    case 'card-3d':
        fromVars = { opacity: 0, y: 40, rotationX: 12, transformPerspective: 1000 };
        break;
    case 'stagger-children':
        fromVars = { opacity: 0, y: 28 };
        break;
}

if (variant === 'stagger-children') {
    const targets = el.children;
    gsap.from(targets, {
        ...fromVars,
        duration,
        delay,
        stagger,
        ease: 'power3.out',
        scrollTrigger: {
            trigger: el,
            start,
            scrub: scrub ? 1 : false,
            toggleActions: 'play none none none',
        },
    });
} else {
    gsap.from(el, {
        ...fromVars,
        duration,
        delay,
        ease: 'power3.out',
        scrollTrigger: {
            trigger: el,
            start,
            scrub: scrub ? 1 : false,
            toggleActions: 'play none none none',
        },
    });
}

}, { scope: containerRef, dependencies: [variant, delay, duration] });

return (
    <div ref={containerRef} className={className} id={id}>
        {children}
    </div>
);
}

```

3.4 Smart Scroll Header Wrapper (components/shared/ScrollHeader.tsx)

Constraint: SiteHeader.tsx must remain completely unmodified (standing project rule — see memory feedback-dont-touch-siteheader). ScrollHeader.tsx is introduced as an outer client wrapper in app/(marketing)/layout.tsx :

```

// components/shared/ScrollHeader.tsx
'use client';
import React, { useRef } from 'react';
import { useGSAP } from '@gsap/react';
import { gsap, ScrollTrigger } from '@lib/gsap';

export function ScrollHeader({ children }: { children: React.ReactNode }) {
  const headerWrapRef = useRef<HTMLDivElement>(null);

  useGSAP(() => {
    const el = headerWrapRef.current;
    if (!el) return;
    let lastScrollY = window.scrollY;

    ScrollTrigger.create({
      start: 'top top',
      end: 'max',
      onUpdate: (self) => {
        const currentScrollY = self.scroll();
        const delta = currentScrollY - lastScrollY;

        if (currentScrollY > 120 && delta > 4) {
          gsap.to(el, { yPercent: -100, duration: 0.35, ease: 'power2.inOut', overwrite: 'auto' });
        } else if (delta < -4 || currentScrollY <= 120) {
          gsap.to(el, {
            yPercent: 0,
            duration: 0.3,
            ease: 'power2.out',
            boxShadow: currentScrollY > 40
              ? '0 10px 25px -5px rgba(0, 0, 0, 0.08), 0 8px 10px -6px rgba(0, 0, 0, 0.04)'
              : 'none',
            overwrite: 'auto',
          });
        }
        lastScrollY = currentScrollY;
      },
    });
  }, { scope: headerWrapRef });

  return (
    <div ref={headerWrapRef} style={{ position: 'sticky', top: 0, zIndex: 100, transition: 'box-shadow 0.2s ease' }}>
      {children}
    </div>
  );
}

```

3.5 Smooth Scroll Provider (components/shared/SmoothScrollProvider.tsx)

Connects Lenis smooth scroll ticker with GSAP ScrollTrigger so pins, scrubs, and parallax match high-refresh desktop monitors:

```
// components/shared/SmoothScrollProvider.tsx
'use client';
import { useEffect } from 'react';
import Lenis from '@studio-freight/lenis';
import { gsap, ScrollTrigger } from '@lib/gsap';

export function SmoothScrollProvider({ children }: { children: React.ReactNode }) {
  useEffect(() => {
    const lenis = new Lenis({
      duration: 1.15,
      easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
      smoothWheel: true,
      touchMultiplier: 1.5,
    });
    lenis.on('scroll', ScrollTrigger.update);

    const updateTicker = (time: number) => lenis.raf(time * 1000);
    gsap.ticker.add(updateTicker);
    gsap.ticker.lagSmoothing(0);

    return () => {
      gsap.ticker.remove(updateTicker);
      lenis.destroy();
    };
  }, []);

  return <>{children}</>;
}
```

4. Section-by-Section GSAP Deep Implementation Blueprint

4.1 Hero Section (components/sections/Hero.tsx) & Map SVG

Visual goal: A cinematic intro on page load where the headline slides up, bullet items with checkmarks draw in sequence, and background map graphics animate.

Create components/sections/HeroMapAnimation.tsx (client component) as the absolute background of Hero.tsx :

```

// components/sections/HeroMapAnimation.tsx
'use client';
import React, { useRef } from 'react';
import { useGSAP } from '@gsap/react';
import { gsap } from '@lib/gsap';

export function HeroMapAnimation() {
  const svgRef = useRef<SVGSVGElement>(null);

  useGSAP(() => {
    if (!svgRef.current) return;
    const svg = svgRef.current;

    const pathway = svg.querySelector('.map-pathway');
    gsap.fromTo(pathway,
      { strokeDashoffset: 400, opacity: 0 },
      { strokeDashoffset: 0, opacity: 0.65, duration: 2.2, ease: 'power2.out', delay: 0.4 }
    );

    const nodes = svg.querySelectorAll('.map-node');
    gsap.fromTo(nodes,
      { scale: 0, transformOrigin: 'center center', opacity: 0 },
      { scale: 1, opacity: 1, duration: 0.8, stagger: 0.15, ease: 'back.out(1.7)', delay: 0.8 }
    );

    nodes.forEach((node, i) => {
      gsap.to(node, {
        y: i % 2 === 0 ? 6 : -6,
        duration: 2.5 + i * 0.4,
        repeat: -1,
        yoyo: true,
        ease: 'sine.inOut',
        delay: 1.2 + i * 0.2,
      });
    });
  }, { scope: svgRef });

  return (
    <div style={{ position: 'absolute', inset: 0, overflow: 'hidden', pointerEvents: 'none', zIndex: 0 }}>
      </* Inlined hero-map-bg.svg with target classes attached to paths & circles */>
    </div>
  );
}

```

Hero content entrance timeline (Hero.tsx):

```

const heroTl = gsap.timeline({ defaults: { ease: 'power3.out' } });
heroTl
  .from('.hero-badge', { opacity: 0, y: -20, duration: 0.6 })
  .from('.hero-heading', { opacity: 0, y: 32, duration: 0.9 }, '-=0.3')
  .from('.hero-description', { opacity: 0, y: 24, duration: 0.7 }, '-=0.5')
  .from('.hero-bullet-item', { opacity: 0, x: -24, stagger: 0.12, duration: 0.6, ease: 'power2.out' }, '-=0.4')
  .from('.hero-cta-group', { opacity: 0, y: 20, scale: 0.97, duration: 0.6 }, '-=0.3');

```

4.2 Problem Section

- 3 source cards stagger in from `y: 40`, `rotationX: 15`, `opacity: 0` (`stagger: 0.15`).
- The arrow-down indicator translates continuously (`y: [0, 8, 0]` , `duration: 1.8` , `repeat: -1`).
- Search mockup: typing simulation or left-to-right highlight sweep.
- Result callouts: the negative box drops in with a subtle shake; the positive box scales up with a soft spring (`scale: 0.95 -> 1.0` , `ease: back.out(1.5)`).

4.3 Problem Solution & Outcome Cards

Collision reveal as the section enters (`start: 'top 75%'`): Before card slides from `x: -50` , After card from `x: +50` , meeting center with `power3.out` . Outcome stat cards stagger with scale pop:

```
gsap.from('.stat-card', {
  scrollTrigger: { trigger: '.stat-card-wrap', start: 'top 80%' },
  scale: 0.88,
  opacity: 0,
  stagger: 0.14,
  duration: 0.75,
  ease: 'back.out(1.4)',
});
```

4.4 Value Comparison

- Animated SVG strikethrough drawn across the agency price via `strokeDashoffset` over 0.6s.
- The £0 card scales up slightly (`scale: 1.04`) with a radial box-shadow pulse in emerald.
- Optional: an animated “amount saved” counter ticking up inside a highlight pill.

4.5 Built To Feel

```
gsap.from('.built-card', {
  scrollTrigger: { trigger: '.built-cards-grid', start: 'top 80%' },
  opacity: 0,
  y: 50,
  rotationY: (index) => (index === 0 ? -8 : index === 2 ? 8 : 0),
  stagger: 0.15,
  duration: 0.85,
  ease: 'power3.out',
});
```

Hover micro-interaction: icon floats up 4px with a spring return.

4.6 Why Sponsored

Central exchange glyph rotates 180° with elastic overshoot on scroll-in:

```
gsap.from('.exchange-glyph-svg', {
  scrollTrigger: { trigger: '.why-sponsored-wrap', start: 'top 75%' },
  rotation: -180,
  scale: 0.4,
  opacity: 0,
  duration: 1.2,
  ease: 'back.out(2)',
});
```

The two exchange panels slide in from opposite sides ($x: -40$ / $x: +40$).

4.7 Scope Comparison

Cascading list-item reveal down each column ($\text{stagger: } 0.05$); “not included” badges enter with a soft bounce.

4.8 After Launch Cards

Already "use client". Upgrade hover state to GSAP `quickTo` for cursor-tracking tilt:

```
const xTo = gsap.quickTo(card, 'x', { duration: 0.4, ease: 'power3' });
const yTo = gsap.quickTo(card, 'y', { duration: 0.4, ease: 'power3' });
// Dynamic 3D tilt based on mouse position relative to card center
```

Staggered entry: $y: 40$, $\text{opacity: } 0$, $\text{stagger: } 0.15$.

4.9 Limited Capacity & Capacity Ring

`LimitedCapacity.tsx` stays an async server component (D1 query untouched). Retire the CSS `@keyframes ui-capacity-ring` (avoids two engines fighting over `strokeDashoffset`) and extract a client component:

```

// components/sections/CapacityRingAnimated.tsx
'use client';
import React, { useRef } from 'react';
import { useGSAP } from '@gsap/react';
import { gsap } from '@lib/gsap';

interface Props {
  accepted: number;
  capacity: number;
  circumference: number;
  targetOffset: number;
}

export function CapacityRingAnimated({ accepted, capacity, circumference, targetOffset }: Props) {
  const ringRef = useRef<SVGCircleElement>(null);
  const numRef = useRef<HTMLSpanElement>(null);

  useGSAP(() => {
    const remaining = Math.max(0, capacity - accepted);

    gsap.fromTo(ringRef.current,
      { strokeDashoffset: circumference },
      {
        strokeDashoffset: targetOffset,
        duration: 1.8,
        ease: 'power3.inOut',
        scrollTrigger: { trigger: ringRef.current, start: 'top 80%', toggleActions: 'play none none none' },
      }
    );

    const counter = { val: 0 };
    gsap.to(counter, {
      val: remaining,
      duration: 1.4,
      ease: 'power2.out',
      scrollTrigger: { trigger: numRef.current, start: 'top 80%' },
      onUpdate: () => {
        if (numRef.current) numRef.current.textContent = Math.round(counter.val).toString();
      },
    });
  }, { scope: ringRef });

  return (
    <div style={{ position: 'relative', display: 'inline-flex', alignItems: 'center', justifyContent: 'center' }}>
      <svg width="120" height="120" viewBox="0 0 120 120">
        <circle cx="60" cy="60" r="50" fill="none" stroke="#e2e8f0" strokeWidth="8" />
        <circle
          ref={ringRef}
          cx="60" cy="60" r="50"
          fill="none"
          stroke="#047857"
          strokeWidth="8"
          strokeDasharray={circumference}
          strokeDashoffset={circumference}
          strokeLinecap="round"
          transform="rotate(-90 60 60)"
        />
      </svg>
      <div style={{ position: 'absolute', textAlign: 'center' }}>
        <span ref={numRef} style={{ fontSize: '1.75rem', fontWeight: 800 }}>0</span>
      </div>
    </div>
  );
}

```

```

        <span style={{ display: 'block', fontSize: '0.75rem', color: '#64748b' }}>left</span>
      </div>
    </div>
  );
}

```

4.10 Process Steps

A scrubbed progress beam runs along the step nodes; the beam fills with an emerald gradient as the user scrolls, and each step's badge activates (scale pop + ring ripple, border brightens) as the beam reaches it:

```

useGSAP(() => {
  const steps = gsap.utils.toArray<HTMLElement>('.process-step-item');

  const tl = gsap.timeline({
    scrollTrigger: {
      trigger: '.process-steps-container',
      start: 'top 70%',
      end: 'bottom 80%',
      scrub: 0.8,
    },
  });
  tl.to('.process-progress-line', { height: '100%', ease: 'none' });

  steps.forEach((step) => {
    ScrollTrigger.create({
      trigger: step,
      start: 'top 65%',
      onEnter: () => gsap.to(step, { opacity: 1, x: 0, duration: 0.5, ease: 'power2.out' }),
    });
  });
});

```

4.11 FAQ Accordion

Native `<details>` / `<summary>` can't animate height smoothly. Replace with `FAQAnimated.tsx` ("use client"): controls open state, tweens `height: 0 -> scrollHeight -> auto` (`duration: 0.4` , `ease: power2.out`), chevron rotates `0→180°`.

4.12 Final CTA

- Subtle aurora/light-sweep gradient drifting behind the text (`x: -20% -> 20%` over 6s, yoyo).
- Magnetic primary CTA button tracking cursor within ~50px via `quickTo` on x/y.
- Reassurance chips float with staggered entry + asynchronous sine-wave drift.

4.13 Header & Footer

- Header: wrapped by `ScrollHeader.tsx` (§3.4) — hides on fast downscroll, reappears instantly on upscroll, gains `backdrop-filter: blur(12px)` after leaving the hero. `SiteHeader.tsx` itself is never edited.
- Footer: top divider expands from `scaleX: 0 -> 1` (center-anchored), columns fade up staggered.

5. Asset Strategy & Motion Graphics Plan

The plan assumes a set of `bg-*.svg` ambient background assets exist under `public/images/` (one per major section) and are currently unused — verify this against the actual `public/` contents at execution time. If present, the intended treatment is a parallax depth layer:

```
gsap.to('.section-ambient-bg', {
  yPercent: -20,
  ease: 'none',
  scrollTrigger: {
    trigger: '.section-wrap',
    start: 'top bottom',
    end: 'bottom top',
    scrub: true,
  },
});
```

Placed as an absolute layer behind content (`pointerEvents: 'none'`), moving at $\sim 0.3x$ scroll speed relative to foreground text.

6. Style & CSS Integration Strategy

6.1 Token Alignment

GSAP scripts must not hardcode arbitrary hex codes — reference the real tokens from `styles/tokens.css` (verify exact names at execution time, e.g. `--color-primary-accessible` , `--color-accent`), not the illustrative names below:

```
export const TOKENS = {
  primary: 'var(--color-primary-accessible, #047857)',
  primaryDark: 'var(--color-primary-accessible-dark, #036450)',
  accent: 'var(--color-accent, #22c55e)',
  easeOut: 'cubic-bezier(0.16, 1, 0.3, 1)',
};
```

6.2 CSS Cleanup Map

When GSAP is introduced, retire the CSS it replaces to avoid two engines fighting over the same transforms:

- `app/globals.css` :retire `.ui-reveal / .ui-reveal-visible` — GSAP takes over transform/opacity orchestration directly.
- `styles/components.css` :retire `@keyframes ui-capacity-ring` (and any `capacity-number` keyframe) once `CapacityRingAnimated.tsx` owns that animation exclusively.

7. Performance, Accessibility & Edge Runtime Safeguards

Cloudflare Workers isolation: Never import GSAP in any file that executes during SSR outside a browser guard or "use client" boundary — Cloudflare Workers throw immediate exceptions if `window / navigator` are referenced during request handling. `lib/gsap.ts` 's `typeof window !==`

'undefined' guard is load-bearing.

Accessibility (prefers-reduced-motion), WCAG 2.2 AA: When the user has prefers-reduced-motion: reduce set — all ScrollTriggers disable transforms (y/x/scale/rotation); opacity snaps to 1 immediately, no stagger delay; hover tilts and magnetic buttons deactivate.

Mobile / low-end device optimization: will-change applied dynamically only during active tweens (onStart / onComplete), not left on permanently. Scrubbed ScrollTriggers on viewports <768px use softer smoothing (scrub: 0.3 instead of 1) to preserve battery and avoid frame drops on mobile WebKit.

8. Phased Implementation Roadmap (For Future Execution)

When instructed to proceed, work in this order:

1. **Foundation** — install gsap , @gsap/react , @studio-freight/lenis ; create lib/gsap.ts ; add the reduced-motion helper.
2. **Wrappers & structural integration** — build GSAPReveal.tsx and ScrollHeader.tsx ; wire ScrollHeader into app/(marketing)/layout.tsx without touching SiteHeader.tsx ; verify zero SSR errors on npm run build .
3. **Hero experience** — inline/tag hero-map-bg.svg in HeroMapAnimation.tsx ; build the entrance timeline in Hero.tsx .
4. **Narrative flow** — problem-card stagger + search mockup scan; value-comparison strikethrough + £0 glow; Built-to-Feel 3D unfold.
5. **Interactive timelines** — extract CapacityRingAnimated.tsx and deprecate the CSS keyframe; implement the scrubbed process-steps timeline.
6. **Polish & conversion** — FAQAnimated.tsx smooth-height accordion; magnetic button physics in FinalCta.tsx .
7. **Performance audit** — test on desktop, iOS Safari, Android Chrome; target a 95+ Lighthouse performance score; validate the Cloudflare Workers deployment (npm run deploy).

Each phase should end in a working, deployable state — not a half-wired one.